Encrypted room file sharing

Temporary rooms, browser encryption and zero-knowledge credential admission.

Organizers distribute files. Participant uploads are excluded.



The room workflow

Create a draft and choose its deadline. Add optional eligibility and location requirements.

Open the room, back up encryption and ownership recovery keys, then send a trusted invitation.

Approve participants or admit them through credential proofs.

Application architecture

React and TypeScript perform encryption and proof generation in the browser.

Express enforces lifecycle, access, proofs and quotas.

PostgreSQL or PGlite stores ciphertext, public keys and operational records.

Privacy boundaries

Private file content, names and decryption keys stay protected by browser cryptography.

Labels, room relationships, sizes, timing and reported location remain visible to the service.

A malicious application origin or compromised browser can steal secrets.

Credential protocol v2

The institution signs hidden credential possession plus department, year and expiry.

Groth16 proves eligibility and derives a room-specific nullifier.

A fresh browser-session challenge prevents copied proofs from taking another session’s seat.

Credential lifecycle

Successful admission consumes the ten-minute challenge. Expiry continues to govern file access.

Issuer-signed room revocations block future proofs and existing membership.

Institution-wide revocation needs the issuer’s worker to apply the scoped policy across affected rooms.

Encrypted distribution

AES-256-GCM encrypts one-MiB chunks and metadata using separate derived keys.

HPKE wraps the file key separately for each eligible recipient.

Immutable retries and the final SHA-256 manifest protect acceptance and authenticated downloads.

Durable upload resume

The browser keeps the source snapshot, plan and immutable encrypted chunks locally.

A reservation commits before encryption. Ciphertext commits before upload.

Interrupted reservations require a fresh key. A reload can safely resume saved ciphertext. Changed recipient keys require a new plan.

Late arrivals and rotation

The organizer grants earlier files by adding recipient envelopes over existing ciphertext.

Key rotation atomically rewraps every existing file key and rejects incomplete replacement sets.

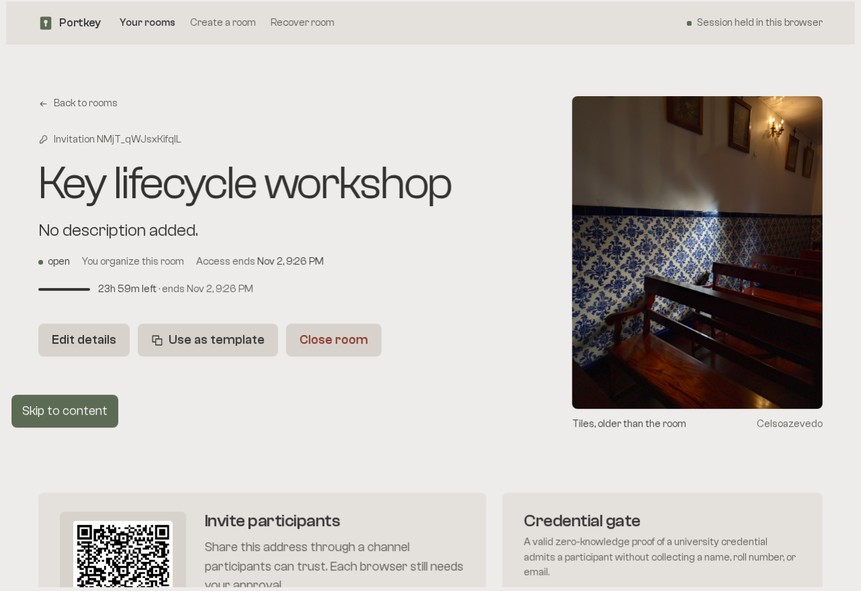
New organizer fingerprints require a fresh trusted invitation and backup.

Ownership recovery

A separate one-time recovery code moves management access into a new browser.

An encrypted key backup restores decryption after recovery.

Previously downloaded files cannot be recalled.



Production operations

PostgreSQL shares rate budgets across API instances.

Docker, HTTPS proxy configuration, protected metrics, cleanup and backup scripts are supplied.

Independent ceremony artifacts and an institution-held issuer remain production prerequisites.

Automated evidence

62 API tests pass on PGlite and 62 on PostgreSQL 17.

9 cases passed across Chromium, Firefox and WebKit engines, including real credential proofs. Actual phones and shipping Safari still need acceptance testing.

Browser evidence covers key recovery, rotation, earlier-file grants, reload/resume and production HTTPS proofs.

250 MiB transfer results

250 chunks preserved all 262,144,000 plaintext bytes.

Publication: 98.2 s. Download and decryption: 34.2 s.

Measured locally with PGlite and headless Chromium. This is not a production throughput guarantee.

Institutional handoff

Supply independent ceremony participation, issuer custody, roster integration and private credential delivery.

Choose the domain and hosting, run CI, and set monitoring and backup policies.

Complete real-device acceptance and independent security review before institutional use.